LAB NO. 10

OPERATOR OVERLOADING

(POLYMORPHISM)

Lab Objectives

Following are the lab objectives:

1. Static (compile-time) polymorphism
2. Operator Overloading

Student ID	**9864**	**Student Name**	**Ali raza**
	Marks	**Obtained Marks**	**Comments**
Task 1	**15**		
Task 2	**15**		
Task 3	**15**		
Total Marks	**45**		

Lab Instructor

Lab Objectives and CLOs Mapping

Lab Objectives	CLOs		
	a	b	C
1			
2			

Instructions

- This is individual Lab work/task.
- Complete this lab work within lab timing.
- Discussion with peers is not allowed.
- You can consult any book, notes & Internet.
- Copy paste from Internet will give you **negative marks**.
- Lab work is divided into small tasks, complete all tasks sequentially.
- Show solution of each lab task to your Lab Instructor.
- In-Lab Exercises/Tasks
- **Write your code at provided space after each question**
- **You need to upload code for all tasks at Google Class.**

LAB TASKS

Task 1 (15 mark)

Create a class **Point** which can store **x** & **y** coordinates of a point exists in plane. Also define following members of class:

- Default constructor
- Parameterized constructor
- Operator +
- Operator -
- Operator *
- Operator /
- Operator !=
- Operator ==

Program:

```
#include <iostream>
using namespace std;

class Point {
private:
   float x, y;

public:
   Point() : x(0), y(0) {
   }
   Point(float a, float b) : x(a), y(b) {
   }

   Point operator+(Point p) {
      return Point(x + p.x, y + p.y);
   }
   Point operator-(Point p) {
      return Point(x - p.x, y - p.y);
   }
   Point operator*(Point p) {
     return Point(x * p.x, y * p.y);
   }
   Point operator/(Point p) {
    return Point(p.x ? x / p.x : 0, p.y ? y / p.y : 0);
   }

   int operator==(Point p) {
      return (x == p.x && y == p.y); }
   int operator!=(Point p) {
      return !(x == p.x && y == p.y);
   }
```

```
    void show() {
        cout << "" << x << endl;
        cout << " " << y << endl;
        cout << "" << endl;
    }
};

int main() {
    Point p1(14, 1), p2(20, 60);

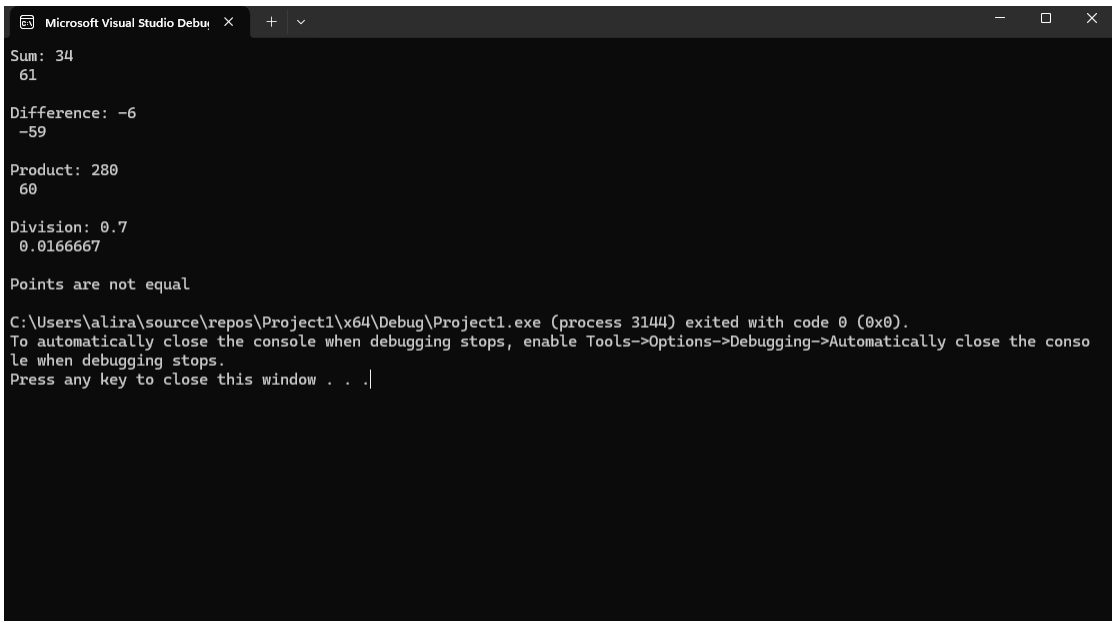
    cout << "Sum: "; (p1 + p2).show();
    cout << "Difference: "; (p1 - p2).show();
    cout << "Product: "; (p1 * p2).show();
    cout << "Division: "; (p1 / p2).show();

    cout << "Points are " << (p1 == p2 ? "equal" : "not equal") << endl;

    return 0;

}
```

Output:



Task 2 (15 mark)

Create a class **RationalNumber** which can store any rational number, expressed in the **P / Q** form. Class also contains following members:

- Default constructor
- Parameterized constructor
- Operator +
- Operator -
- Operator *
- Operator /
- Operator !=
- Operator ==
- Operator <
- Operator >
- Operator <=
- Operator >=

Program :

```
#include <iostream>
using namespace std;
class RationalNumber {
private:
   int nominator, denominator;

public:
   RationalNumber() : nominator(0), denominator(1) {}

   RationalNumber(int num, int den) {
      nominator = num;
      denominator = den + (den == 0);
   }

   RationalNumber operator+(RationalNumber r) {
      return RationalNumber(nominator * r.denominator + r.nominator * denominator,
         denominator * r.denominator);
   }

   RationalNumber operator-(RationalNumber r) {
      return RationalNumber(nominator * r.denominator - r.nominator * denominator,
         denominator * r.denominator);
   }

       RationalNumber operator*(RationalNumber r) {
      return RationalNumber(nominator * r.nominator,
         denominator * r.denominator);
   }
```

```
  RationalNumber operator/(RationalNumber r) {
        return RationalNumber(nominator * r.denominator,
         denominator * r.nominator + (r.nominator == 0));
    }

    int operator==(RationalNumber r) {
        return (nominator * r.denominator == r.nominator * denominator);
    }

      int operator!=(RationalNumber r) {
        return !(nominator * r.denominator == r.nominator * denominator);
    }

    int operator<(RationalNumber r) {
        return (nominator * r.denominator < r.nominator * denominator);
    }

       int operator>(RationalNumber r) {
         return (nominator * r.denominator > r.nominator * denominator);
    }

        int operator<=(RationalNumber r) {
         return !(nominator * r.denominator > r.nominator * denominator);
    }

  int operator>=(RationalNumber r) {
          return !(nominator * r.denominator < r.nominator * denominator);
    }

    void show() {
        cout << nominator << " / " << denominator << endl;
    }
};

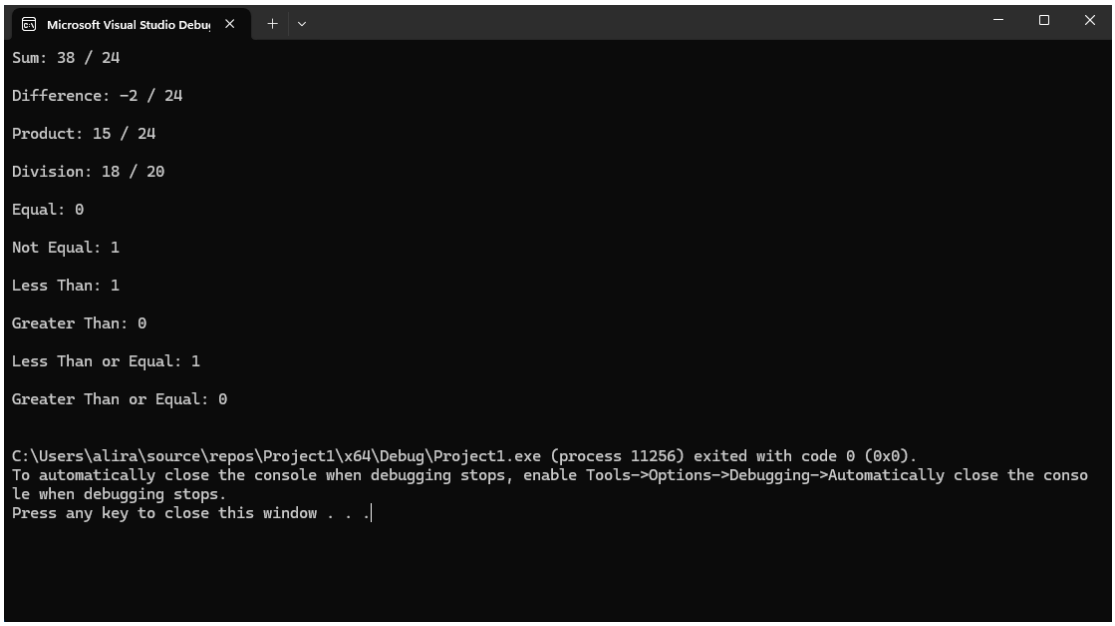
int main() {
    RationalNumber r1(3, 4), r2(5, 6);

    cout << "Sum: "; (r1 + r2).show();
    cout << "" << endl;
        cout << "Difference: "; (r1 - r2).show();
     cout << "" << endl;
        cout << "Product: "; (r1 * r2).show();
     cout << "" << endl;
        cout << "Division: "; (r1 / r2).show();
       cout << "" << endl;
        cout << "Equal: " << (r1 == r2) << endl;
         cout << "" << endl;
        cout << "Not Equal: " << (r1 != r2) << endl;
        cout << "" << endl;
        cout << "Less Than: " << (r1 < r2) << endl;
  cout << "" << endl;
        cout << "Greater Than: " << (r1 > r2) << endl;
       cout << "" << endl;
        cout << "Less Than or Equal to : " << (r1 <= r2) << endl;
      cout << "" << endl;
        cout << "Greater Than or Equal to: " << (r1 >= r2) << endl;
      cout << "" << endl;

    return 0;

}
```

Output:


Microsoft Visual Studio Debu
Sum: 38 / 24
Difference: -2 / 24
Product: 15 / 24
Division: 18 / 20
Equal: 0
Not Equal: 1
Less Than: 1
Greater Than: 0
Less Than or Equal: 1
Greater Than or Equal: 0
C:\Users\alira\source\repos\Project1\x64\Debug\Project1.exe (process 11256) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .

Task 3 (15 mark)

Create a class **Distance** which can store any distance in the form of **feet** and **inches**, e.g. 9'6'' (9 feet and 6 inches). Class also contains following members:

- Default constructor
- Parameterized constructor
- Operator +
- Operator -
- Operator !=
- Operator ==
- Operator <
- Operator >
- Operator <=
- Operator >=

Program :

```
#include <iostream>
using namespace std;

class Distance {
private:
   int feet;
   int inches;

public:

   Distance() {
      feet = 0;
      inches = 0;
   }

   Distance(int f, int i) {
      feet = f;
      inches = i;
   }


   Distance operator+(Distance d) {
      return Distance(feet + d.feet, inches + d.inches);
   }


   Distance operator-(Distance d) {
      return Distance(feet - d.feet, inches - d.inches);
   }
```

```
    int operator==(Distance d) {
        return (feet == d.feet && inches == d.inches) ? 1 : 0;
    }

    int operator!=(Distance d) {
        return (feet != d.feet || inches != d.inches) ? 1 : 0;
    }

    int operator<(Distance d) {
        return (feet < d.feet || (feet == d.feet && inches < d.inches)) ? 1 : 0;
    }

    int operator>(Distance d) {
        return (feet > d.feet || (feet == d.feet && inches > d.inches)) ? 1 : 0;
    }


    int operator<=(Distance d) {
        return (*this < d || *this == d) ? 1 : 0;
    }

    int operator>=(Distance d) {
        return (*this > d || *this == d) ? 1 : 0;
    }

    void show() {
        cout << feet << "" << inches << "" << endl;
    }
};

int main() {
    Distance d1(5, 8);
    Distance d2(3, 11);

    Distance sum = d1 + d2;
    Distance diff = d1 - d2;

    cout << "d1:"; d1.show();
    cout << "" << endl;
    cout << "d2:"; d2.show();
    cout << "" << endl;
    cout << "Sum:"; sum.show();
    cout << "" << endl;
    cout << "Difference:"; diff.show();
    cout << "" << endl;

    cout << "d1 == d2: " << (d1 == d2) << endl;
    cout << "" << endl;
    cout << "d1 != d2: " << (d1 != d2) << endl;
    cout << "" << endl;
    cout << "d1 < d2: " << (d1 < d2) << endl;
    cout << "" << endl;
    cout << "d1 > d2: " << (d1 > d2) << endl;
    cout << "" << endl;
    cout << "d1 <= d2: " << (d1 <= d2) << endl;
    cout << "" << endl;
    cout << "d1 >= d2: " << (d1 >= d2) << endl;
    cout << "" << endl;

    return 0;
}
```

Output:


Microsoft Visual Studio Debu
d1:58
d2:311
Sum:819
Difference:2-3
d1 == d2: 0
d1 != d2: 1
d1 < d2: 0
d1 > d2: 1
d1 <= d2: 0
d1 >= d2: 1